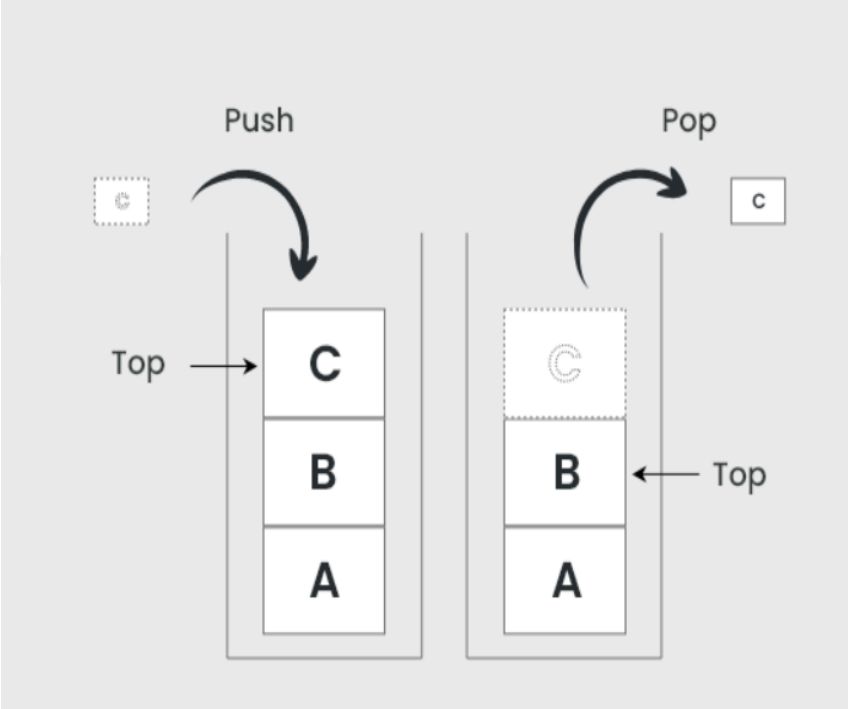
OBJECT ORIENTED PROGRAMMING USING C++

Study Materials

Stack:

- A Stack is a linear data structure that follows the LIFO (Last-In-First-Out) principle.
- Stack has one end, whereas the Queue has two ends (front and rear).
- It contains only one pointer top pointer pointing to the topmost element of the stack.
- Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack.
- In other words, a stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.

Stack representation:



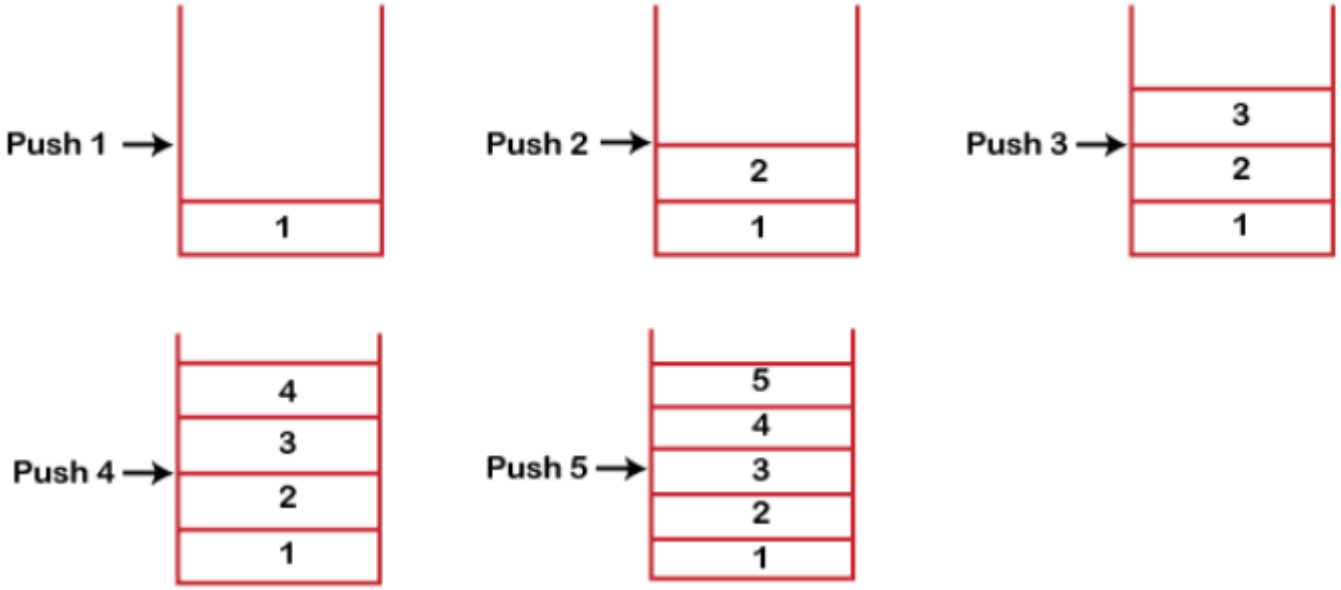
key points:

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.

Working of Stack:

- Stack works on the LIFO pattern. As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.
- Suppose we want to store the elements in a stack and let's assume that stack is empty.
- We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.
- Since our stack is full as the size of the stack is 5.
- In the above cases, we can observe that it goes from the top to the bottom when we were entering the new element in the stack.
- The stack gets filled up from the bottom to the top.

Representation of Stack:



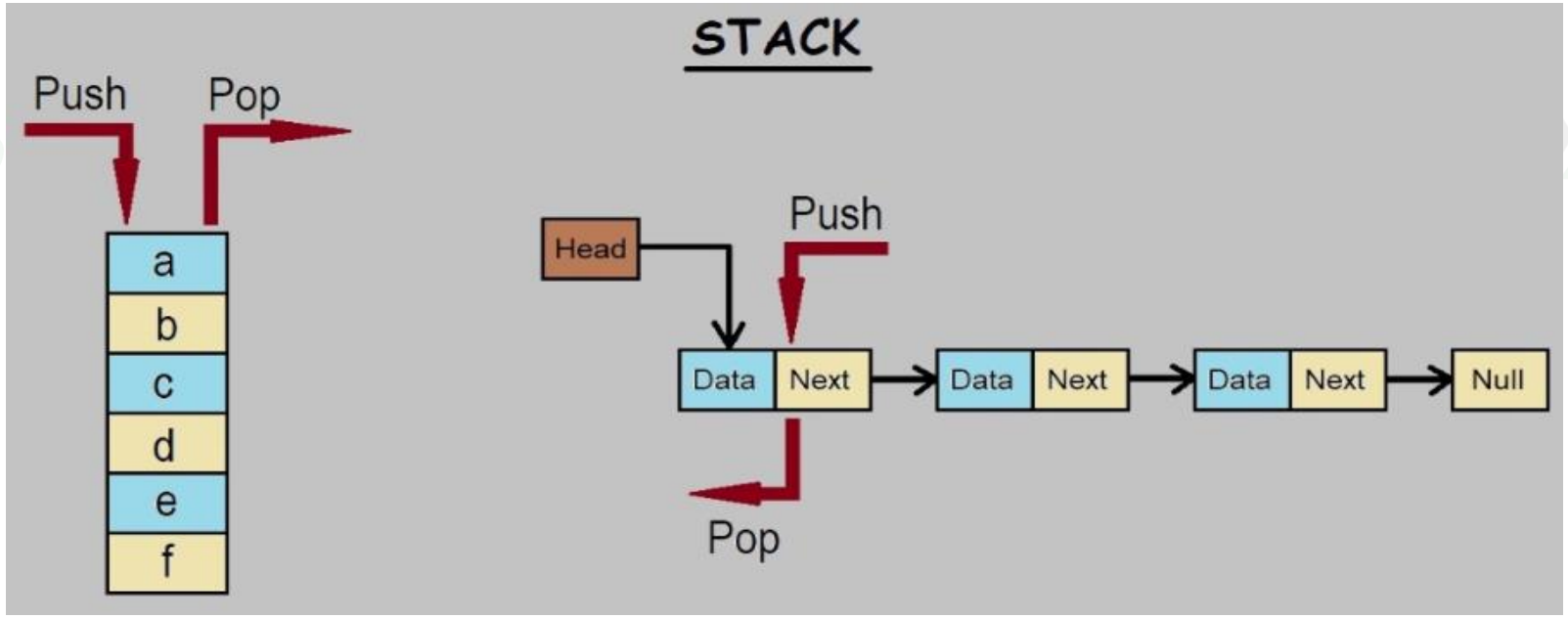
Basic Operations:

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if the stack is empty else false.
- **size()** returns the size of the stack.

Linked list implementation:

- Instead of using array, we can also use linked list to implement stack.
- Linked list allocates the memory dynamically.
- However, time complexity in both the scenario is same for all the operations push, pop and peek.
- In linked list implementation of stack, the nodes are maintained non-contiguously in the memory.
- Each node contains a pointer to its immediate successor node in the stack.
- Stack is said to be overflown if the space left in the memory heap is not enough to create a node.

Linked list implementation:



Array implementation:

- An array implementation, the stack is formed by using the array.
- All the operations regarding the stack are performed using arrays.

- **Step-by-step approach:**
- Initialize an array to represent the stack.
- Use the end of the array to represent the top of the stack.
- Implement push (add to end), pop (remove from the end), and peek (check end) operations, ensuring to handle empty and full stack conditions.

Array implementation: